

Министерство науки и высшего образования Российской Федерации  
Федеральное государственное автономное образовательное учреждение  
высшего образования

«Национальный исследовательский университет ИТМО»

Факультет программной инженерии и компьютерной техники

Отчёт по домашней работе № 4

по дисциплине «Расширенный курс по бэкенд-разработке в среде Node.js»

**Технический дизайн микросервисной архитектуры сервиса  
«Lecter's Recipes»**

Вариант 5

Выполнил: студент группы БР1.1  
Кошкарев Кирилл Павлович

Проверил:

---

Санкт-Петербург, 2026

## 1. Цель работы

Спроектировать разделение монолитного приложения ЛР1 на микросервисы по принципу database-per-service: определить границы сервисов и их базы данных, описать способы взаимодействия, спроектировать внутренние эндпоинты в формате OpenAPI и задокументировать возможные ошибки.

## 2. Принцип выделения границ

Границы проведены по зонам ответственности предметной области, а не по техническим слоям. Признаки, по которым сгруппированы сущности: общий жизненный цикл данных, общий владелец изменений и совместное использование в одних и тех же транзакциях.

Из монолита выделены три сервиса и шлюз:

Таблица 1 — Состав микросервисной архитектуры

| Сервис         | Порт | База данных     | Зона ответственности                                                                                       |
|----------------|------|-----------------|------------------------------------------------------------------------------------------------------------|
| api-gateway    | 3000 | —               | единая точка входа /api/v1/*, проверка JWT, маршрутизация, агрегация ответов, публикация Swagger и витрины |
| user-service   | 3001 | lecters_users   | регистрация, вход, выпуск JWT, профили, граф подписок                                                      |
| recipe-service | 3002 | lecters_recipes | рецепты, состав, шаги, медиа, справочники, поиск и фильтрация                                              |
| social-service | 3003 | lecters_social  | лайки, избранное, комментарии, локальные проекции пользователей и рецептов                                 |

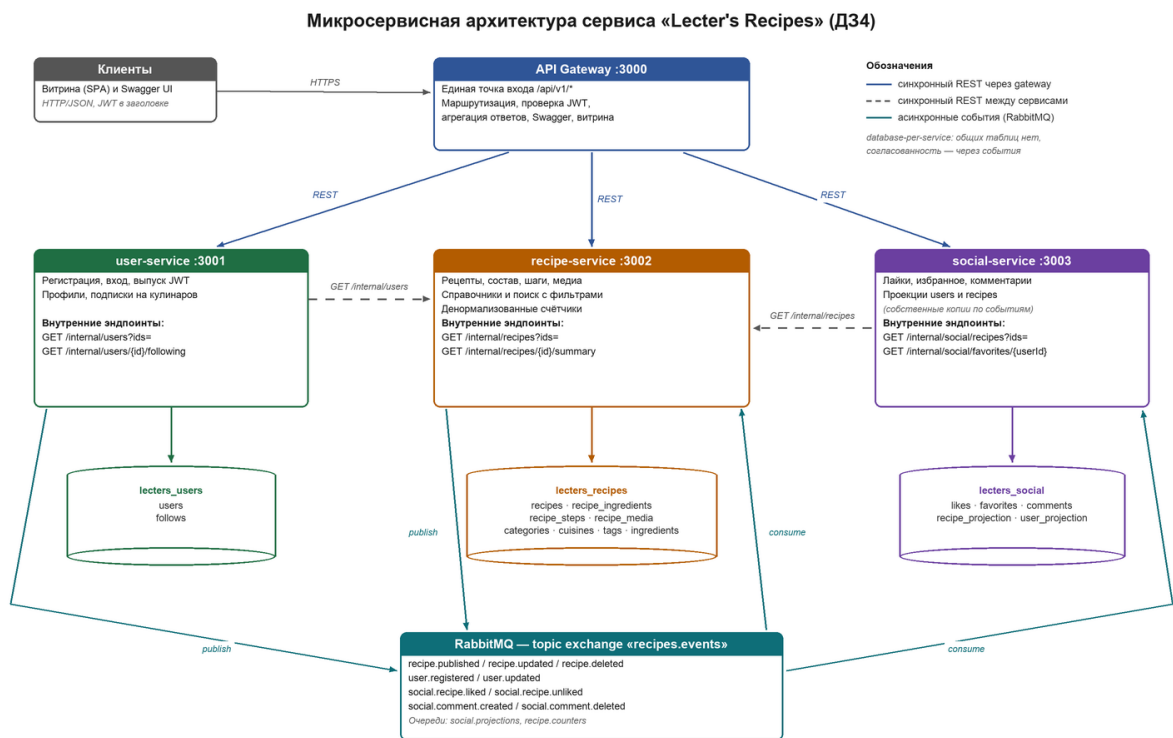


Рисунок 1 — микросервисная архитектура: сервисы, шлюз, брокер и потоки данных

### 3. Разделение базы данных

Единая база монолита разделяется на три независимые базы. Общих таблиц нет, межбазовых внешних ключей нет: ссылки на сущности чужого сервиса хранятся как обычные числовые поля (authorId в рецептах, userId и recipeId в лайках). Ссылочная целостность обеспечивается на уровне приложения — проверками при записи и событиями при удалении.

## Разделение базы данных по сервисам (database-per-service)

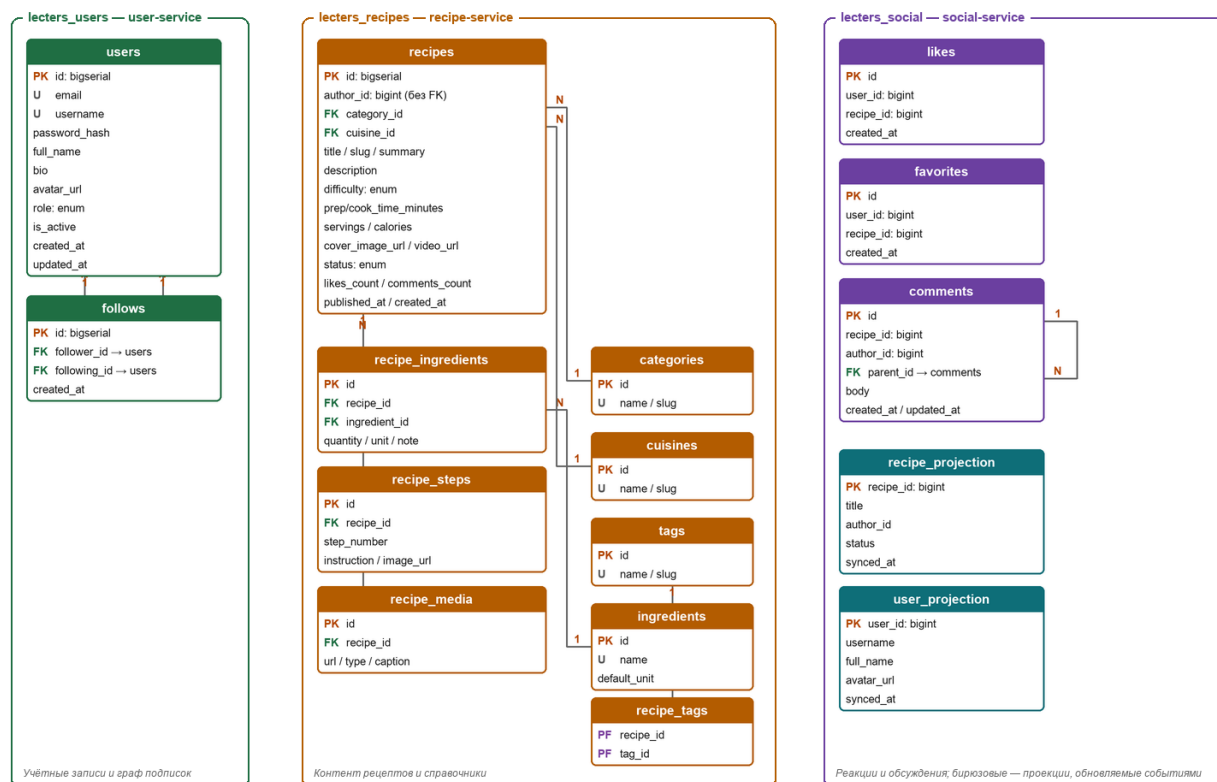


Рисунок 2 — разделение базы данных по сервисам (database-per-service)

В базе social-service появляются две новые таблицы — проекции `recipe_projection` и `user_projection`. Они хранят минимум сведений о чужих сущностях (название рецепта, автор, статус; имя пользователя и аватар) и наполняются событиями. Проекция позволяет отдавать комментарии с именами авторов и проверять существование рецепта, не обращаясь к соседнему сервису на каждый запрос.

## 4. Способы взаимодействия

Применяются два вида взаимодействия — синхронное для запросов на чтение в момент обработки пользовательского запроса и асинхронное для распространения изменений.

Таблица 2 — Виды межсервисного взаимодействия

| Вид        | Когда применяется                                                                                             | Механизм                                                                                          | Поведение при отказе                                                                           |
|------------|---------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------|
| Синхронный | Шлюзу нужно собрать ответ из данных нескольких сервисов; сервису нужно проверить существование чужой сущности | HTTP-запрос к <code>/internal/*</code> с заголовком <code>X-Internal-Token</code> , таймаут 3–5 с | Для второстепенных данных — деградация: ответ отдаётся без них. Для обязательных — 503 или 504 |

| Вид         | Когда применяется                                            | Механизм                                          | Поведение при отказе                                                                                                       |
|-------------|--------------------------------------------------------------|---------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------|
| Асинхронный | Изменилось состояние, о котором нужно узнать другим сервисам | Событие в RabbitMQ, topic-exchange recipes.events | Публикация не блокирует пользовательский сценарий; расхождение устраняется синхронным вызовом или повторной синхронизацией |

Аутентификация не выделена в отдельный вызов: JWT подписывается общим секретом, поэтому каждый сервис проверяет токен самостоятельно. Это убирает обращение к user-service на каждый защищённый запрос ценой того, что секрет должен распространяться на все сервисы.

## 5. Внутренние эндпоинты

Внутренние эндпоинты живут под префиксом /internal и не публикуются наружу: шлюз проксирует только /api/v1/\*. Вызовы авторизуются общим секретом внутренней сети в заголовке X-Internal-Token. Полная спецификация — в файле openapi-internal.yaml.

Таблица 3 — Эндпоинты межсервисного взаимодействия

| Эндпоинт                                | Владелец       | Назначение                                      | Потребители             |
|-----------------------------------------|----------------|-------------------------------------------------|-------------------------|
| GET /internal/users?ids=                | user-service   | массовая выборка профилей (до 100 за вызов)     | gateway, social-service |
| GET /internal/users/{id}/following      | user-service   | идентификаторы подписок для ленты               | gateway                 |
| GET /internal/users/{id}/exists         | user-service   | проверка существования пользователя             | social-service          |
| GET /internal/recipes?ids=              | recipe-service | массовая выборка карточек рецептов              | gateway, social-service |
| GET /internal/recipes/{id}/summary      | recipe-service | сводка о рецепте, если проекция отстала         | social-service          |
| GET /internal/recipes/by-authors        | recipe-service | публикации перечисленных авторов (лента)        | gateway                 |
| GET /internal/social/recipes?ids=       | social-service | счётчики и состояние зрителя по списку рецептов | gateway                 |
| GET /internal/social/favorites/{userId} | social-service | идентификаторы сохранённых рецептов             | gateway                 |
| GET /internal/social/health             | social-service | состояние сервиса, брокера и проекций           | gateway, мониторинг     |

Ключевое правило контракта: внутренние ответы не содержат вложенных сущностей чужих сервисов. Например, /internal/recipes отдаёт карточку с полем authorId, но без объекта author — сшиванием занимается

вызывающая сторона. Это исключает циклические вызовы между сервисами.

```
/internal/social/recipes:
  get:
    summary: Счётчики реакций и состояние текущего пользователя по списку рецептов
    parameters:
      - { name: ids, in: query, required: true, schema: { type: string } }
      - { name: userId, in: query, schema: { type: integer, format: int64 } }
    responses:
      '200':
        description: Состояние реакций
        content:
          application/json:
            example:
              data:
                - { recipeId: 12, likesCount: 42, commentsCount: 7,
                    liked: true, favorite: false }
      '400': { $ref: '#/components/responses/BadRequest' }
      '401': { $ref: '#/components/responses/Unauthorized' }
```

Листинг 3 — фрагмент спецификации внутреннего API

## 6. Ошибки межсервисных вызовов

Таблица 4 — Ошибки внутреннего API и реакция на них

| Код                     | Причина                                     | Реакция вызывающей стороны                                               |
|-------------------------|---------------------------------------------|--------------------------------------------------------------------------|
| 400 VALIDATION_ERROR    | Не передан ids или их больше 100            | Ошибка программиста: запрос разбивается на порции по 100 идентификаторов |
| 401 UNAUTHORIZED        | Неверный или отсутствующий X-Internal-Token | Вызов не повторяется, пишется ошибка конфигурации                        |
| 404 NOT_FOUND           | Сущность отсутствует у владельца            | Для проверки существования — отказ операции с 404 наружу                 |
| 503 SERVICE_UNAVAILABLE | Сервис-владелец недоступен                  | Для второстепенных данных — деградация, для обязательных — 503 наружу    |
| 504 UPSTREAM_TIMEOUT    | Превышен таймаут ответа                     | Аналогично 503; шлюз не держит пользователя дольше 5 с                   |

## 7. Асинхронные события

Обмен событиями построен на topic-exchange recipes.events. Каждый потребитель объявляет собственную durable-очередь и привязывает её по шаблону ключа маршрутизации. Контракт зафиксирован в файле events.yaml.

Таблица 5 — Каталог событий

| Событие          | Издатель       | Потребитель    | Следствие                          |
|------------------|----------------|----------------|------------------------------------|
| user.registered  | user-service   | social-service | создаётся запись в user_projection |
| user.updated     | user-service   | social-service | обновляется user_projection        |
| recipe.published | recipe-service | social-service | создаётся recipe_projection        |
| recipe.updated   | recipe-service | social-service | обновляется recipe_projection      |

| Событие                | Издатель       | Потребитель    | Следствие                                         |
|------------------------|----------------|----------------|---------------------------------------------------|
| recipe.deleted         | recipe-service | social-service | каскадно удаляются лайки, избранное и комментарии |
| social.recipe.liked    | social-service | recipe-service | likes_count += 1                                  |
| social.recipe.unliked  | social-service | recipe-service | likes_count -= 1                                  |
| social.comment.created | social-service | recipe-service | comments_count += 1                               |
| social.comment.deleted | social-service | recipe-service | comments_count -= 1                               |

Все сообщения передаются в едином конверте с полями `eventId`, `type`, `occurredAt`, `producer`, `version` и `payload`. Поле `eventId` служит ключом идемпотентности: потребитель хранит обработанные идентификаторы и игнорирует повторную доставку.

## 8. Согласованность данных

Отказ от межбазовых внешних ключей означает отказ от немедленной согласованности. Принятая модель — согласованность в конечном счёте со следующими гарантиями:

- **доставка не менее одного раза**: потребитель обязан быть идемпотентным; счётчики изменяются выражением вида «поле = поле + дельта», а не установкой значения;
- **порядок** гарантируется только внутри одной очереди; проекции хранят метку `synced_at` и не применяют событие старше уже сохранённого состояния;
- **отставание проекции** компенсируется синхронным вызовом: если рецепта ещё нет в проекции `social-service`, выполняется запрос `/internal/recipes/{id}/summary`, и лайк не теряется;
- **неудачная обработка** не закидывает очередь: после трёх попыток сообщение уходит в `dead-letter`-очередь для разбора.

## 9. План разделения монолита

- 1) Создать три пустые базы данных с той же локалью, что и у монолита.
- 2) Вынести `user-service`: перенести сущности `users` и `follows`, логику регистрации, входа и подписок; сервис становится единственным издателем JWT.
- 3) Вынести `recipe-service`: перенести рецепты, справочники и поиск; заменить связь `recipe.author` на числовое поле `authorId`.

- 4) Вынести social-service: перенести лайки, избранное и комментарии; добавить таблицы проекций и журнал обработанных событий.
- 5) Поднять API Gateway: сначала как прозрачный прокси, затем добавить агрегирующие маршруты витрины, ленты и сохранённого.
- 6) Подключить RabbitMQ и перевести обновление счётчиков и проекций на события (Д35).
- 7) Проверить, что публичный контракт не изменился: коллекция Postman из Д33 должна проходить против шлюза без правок.

## 10. Оценка решения

Плюсы выбранного разделения: сервисы можно разрабатывать, разворачивать и масштабировать независимо; отказ social-service не мешает читать рецепты; нагрузка на поиск изолирована в recipe-service.

Издержки, которые принимаются осознанно: усложняется отладка (один пользовательский запрос порождает несколько межсервисных), появляется дублирование данных в проекциях, а согласованность становится отложенной. Для учебного проекта такого масштаба монолит остаётся проще, поэтому разделение выполняется прежде всего как отработка архитектурного приёма.

## 11. Выводы

Монолит разделён на три сервиса и шлюз по принципу database-per-service. Описаны 9 внутренних эндпоинтов и 9 типов событий с указанием издателей, потребителей и последствий, задокументированы ошибки межсервисных вызовов и правила деградации. Полученный документ служит планом реализации для ЛР2 и Д35.